

TRÌNH BIÊN DỊCH

Chương 2. Phân tích cú pháp-LL(1)

Phạm Nguyễn Khang
BM. Khoa học máy tính

pnkhang@cit.ctu.edu.vn

CẦN THƠ, 12 - 2012

```
#include ...  
int main() {  
    ...  
}
```



Phân tích từ vựng

Phân tích cú pháp

Phân tích ngữ nghĩa

Sinh mã trung gian

Tối ưu mã trung gian

Sinh mã

Tối ưu



```
0001001000  
1011011100  
1100101111  
1000011111
```

Phân tích từ trên xuống

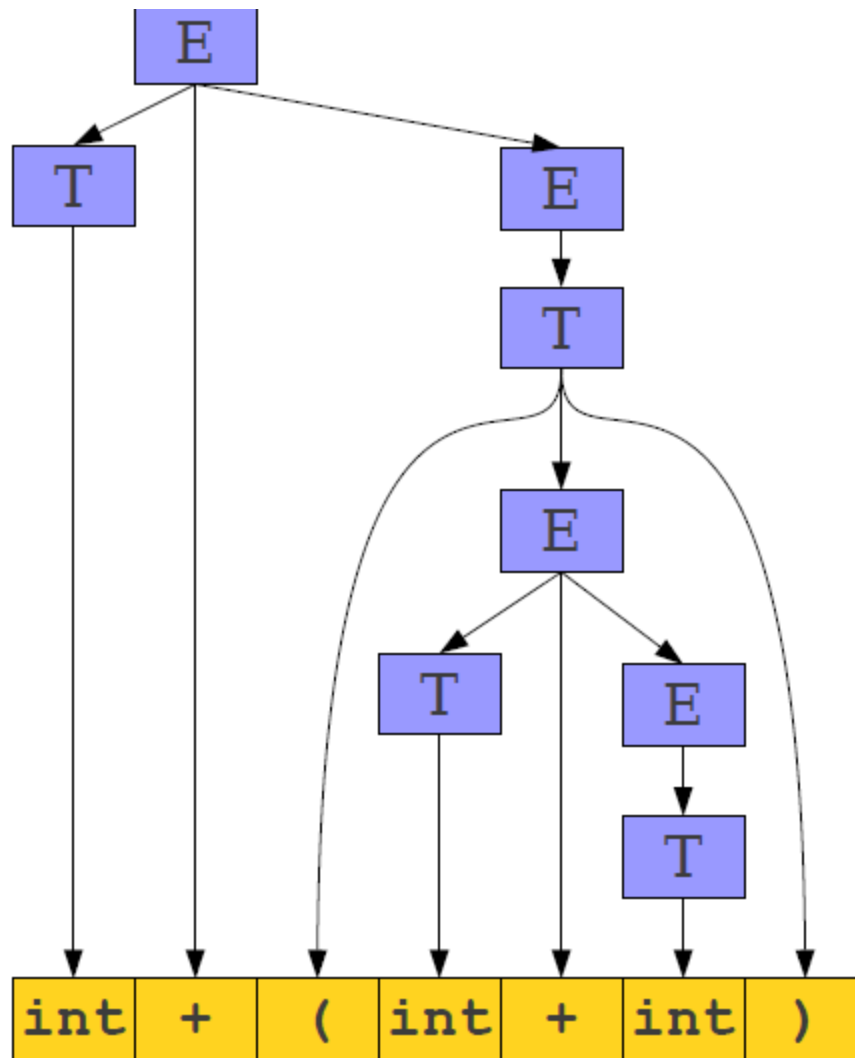
- Nội dung tuần trước:
 - Mục tiêu của phân tích cú pháp: xây dựng cấu trúc của chương trình.
 - Ý tưởng: Sử dụng văn phạm phi ngữ cảnh để mô tả ngôn ngữ lập trình.
 - Với chuỗi các tokens sinh ra từ bộ phân tích từ vựng, ta tìm một **cây phân tích cú pháp sinh ra chuỗi các tokens này**.
 - Quá trình xây dựng cây cú pháp này được gọi là **phân tích cú pháp**.

Phương pháp phân tích cú pháp

- Phân tích từ trên xuống (top-down parsing)
 - Bắt đầu từ kí hiệu bắt đầu, cố gắng đoán luật sinh nào được sử dụng để sinh ra chương trình đầu vào.
- Phân tích từ dưới lên (bottom-up parsing)
 - Bắt đầu từ chương trình đầu vào, cố gắng áp dụng các luật sinh theo chiều ngược để biến đổi chương trình thành ký hiệu bắt đầu.

Phân tích từ trên xuống

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$



Những khó khăn

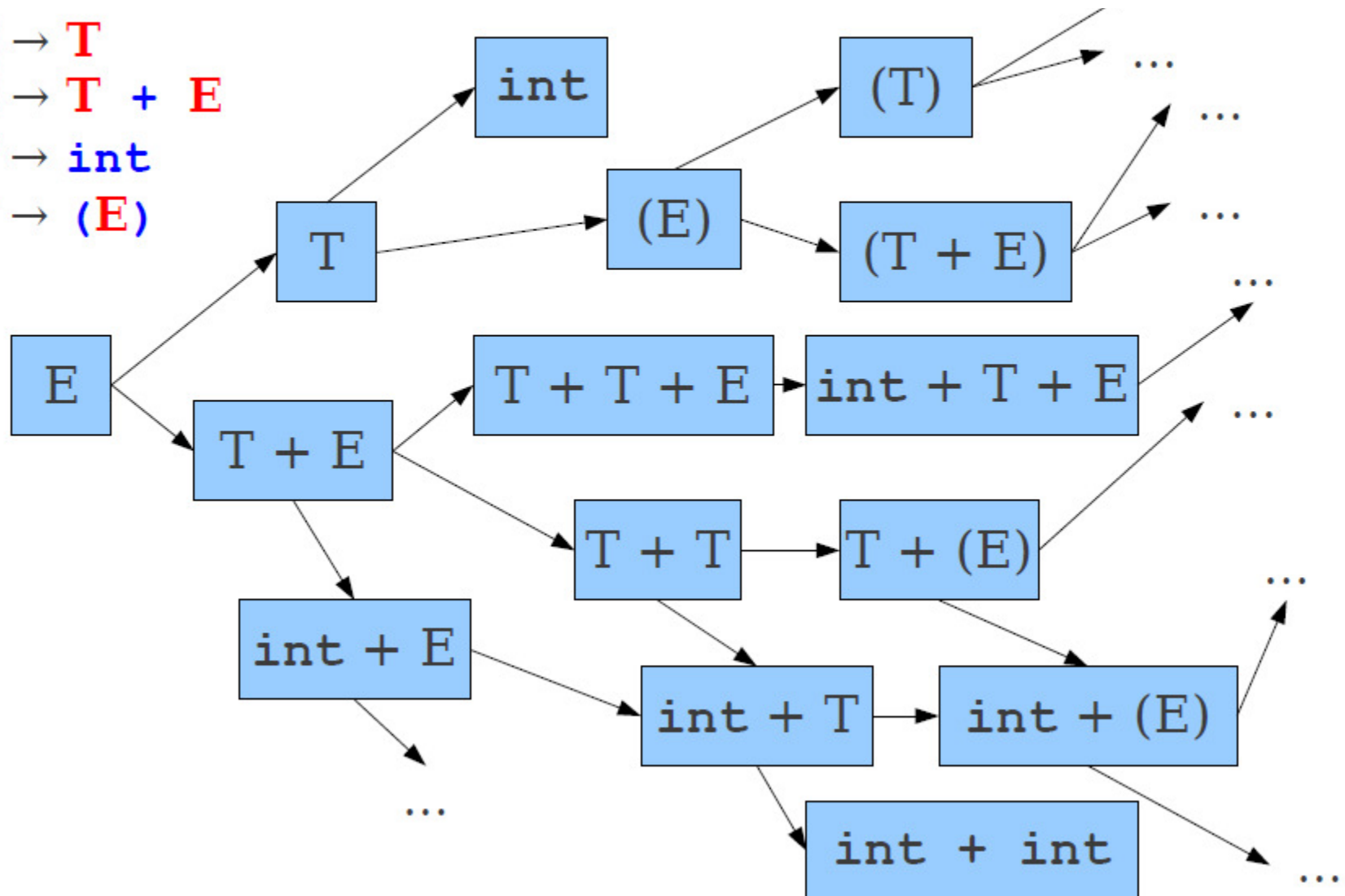
- Phân tích từ trên xuống bắt đầu với không có thông tin
 - Bắt đầu chỉ với ký hiệu bắt đầu của văn phạm.
- Làm sao ta biết phải áp dụng luật nào ?
- Một cách tổng quát, ta không thể biết được.
 - Có một số văn phạm ta chỉ có thể đoán và nếu sai ta quay lui làm lại.
- Nếu như phải đoán luật nào phải được áp dụng, ta sẽ đoán như thế nào ?

Phân tích cú pháp như tìm kiếm

- Ý tưởng: xem bài toán **phân tích cú pháp như bài toán tìm kiếm trên đồ thị**.
- Mỗi nút được biểu diễn dưới **dạng câu** (sentential form), một chuỗi các kí hiệu kết thúc và chưa kết thúc có thể được dẫn xuất ra từ kí hiệu bắt đầu.
- Cung nối từ nút α đến nút $\beta \Leftrightarrow \alpha \Rightarrow \beta$
 - α dẫn xuất ra β

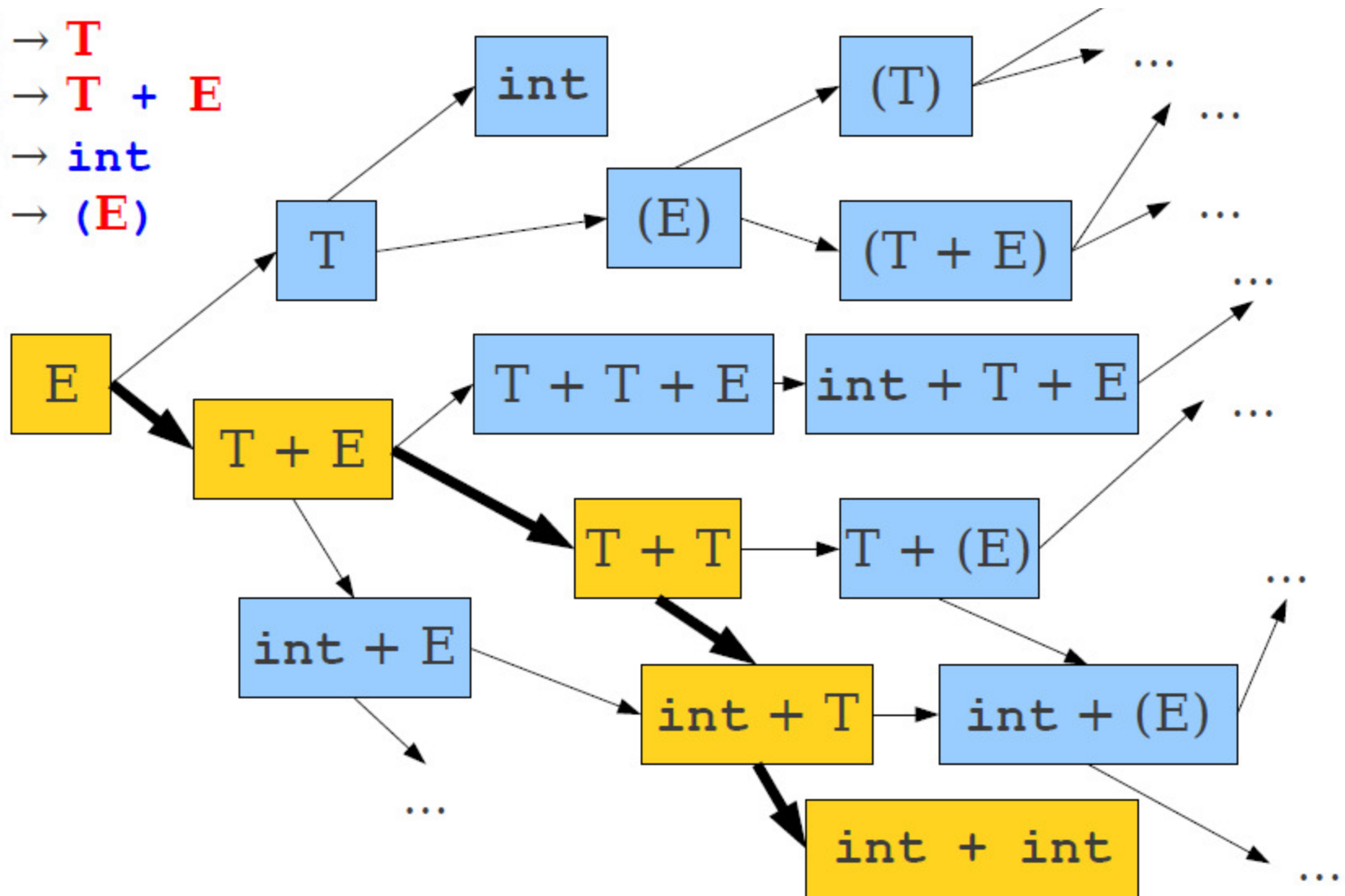
Phân tích cú pháp như tìm kiếm

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$



Phân tích cú pháp như tìm kiếm

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$



Giải thuật phân tích từ trên xuống đầu tiên

- **Tìm kiếm theo chiều rộng (Breadth-First Search)**

- Sử dụng một danh sách (hàng đợi) các **câu**, khởi tạo danh sách này chứa nút **S**
- while (hàng đợi chưa rỗng)
 - Lấy **câu** đầu tiên ra khỏi hàng đợi
 - Nếu nó khớp với chương trình đầu vào → kết thúc
 - Ngược lại, với mỗi câu được sinh ra từ câu hiện tại, thêm nó vào trong hàng đợi.
- Khi vòng lặp while kết thúc ta có thể truy vết lại để xây dựng cây cú pháp.

Phân tích theo kiểu BFS

Worklist

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

Phân tích theo kiểu BFS

Worklist

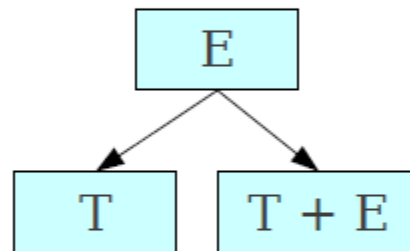
E

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

Phân tích theo kiểu BFS

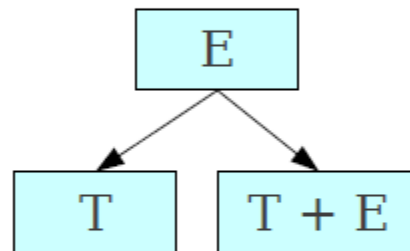
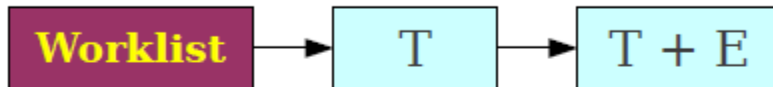
Worklist



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

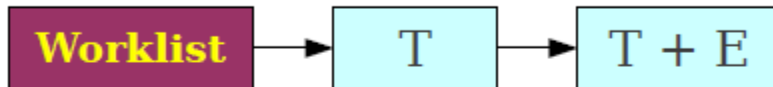
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

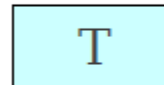
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

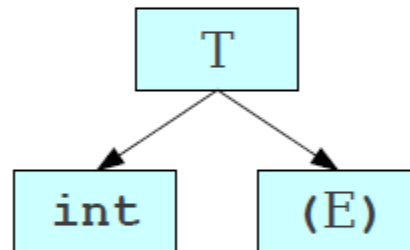
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

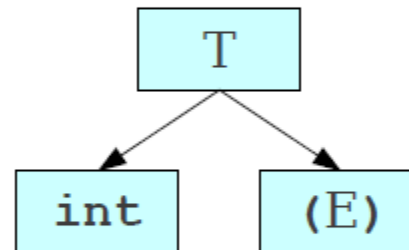
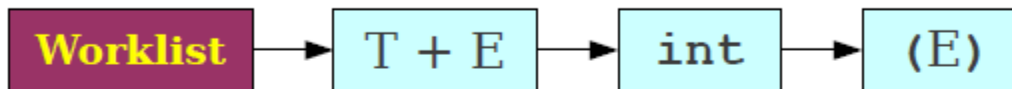
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

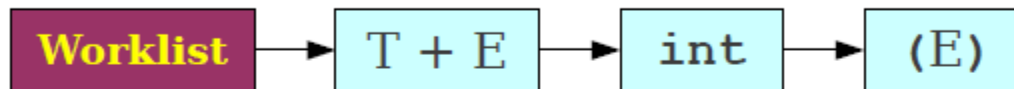
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

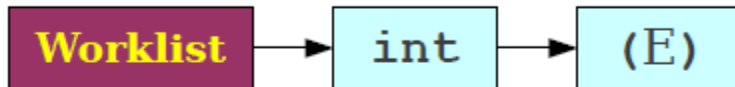
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

Phân tích theo kiểu BFS

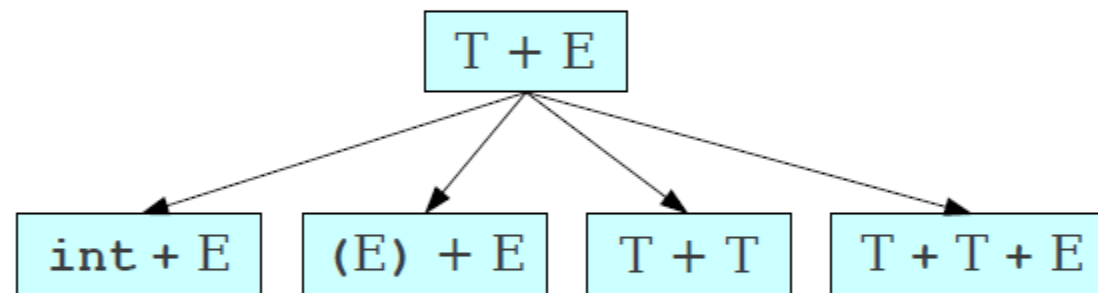
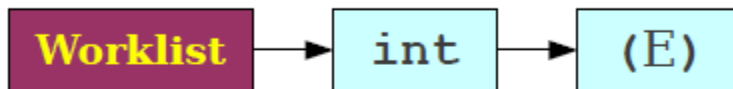


T + E

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

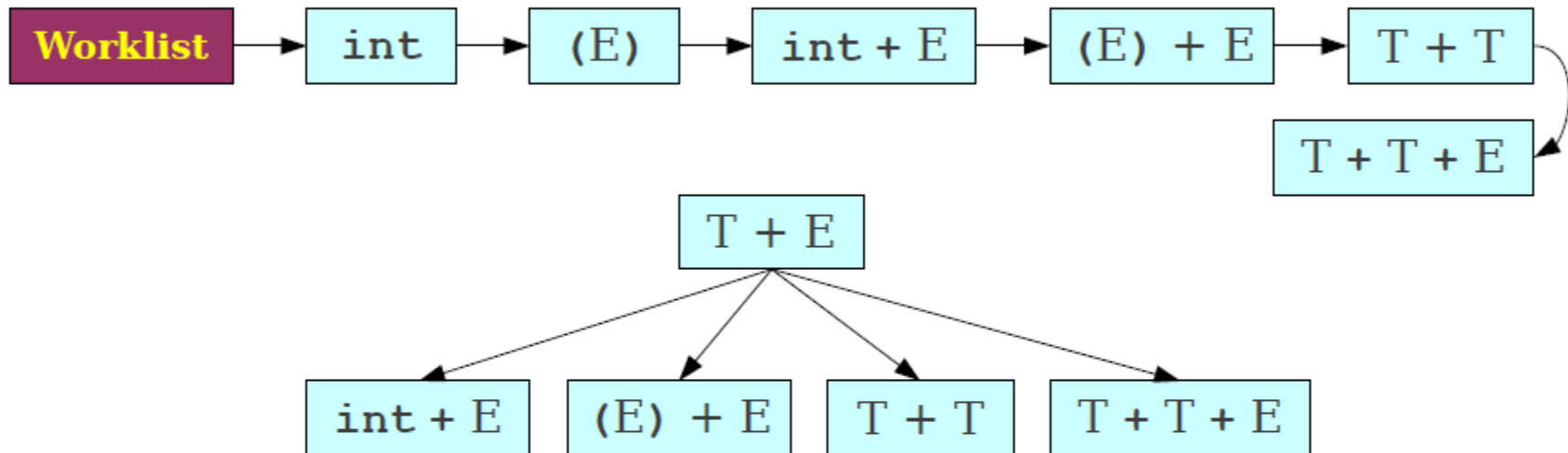
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

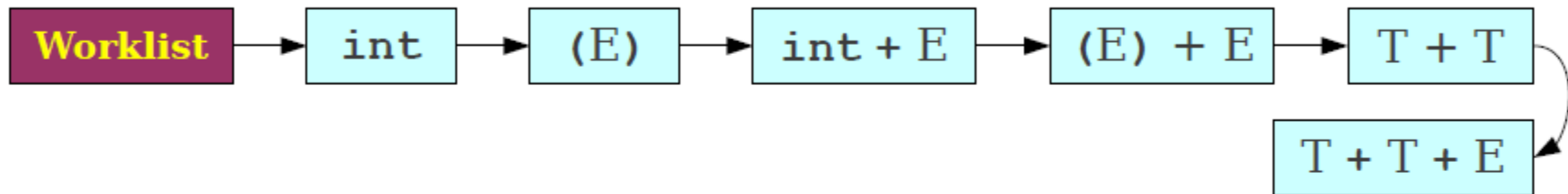
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

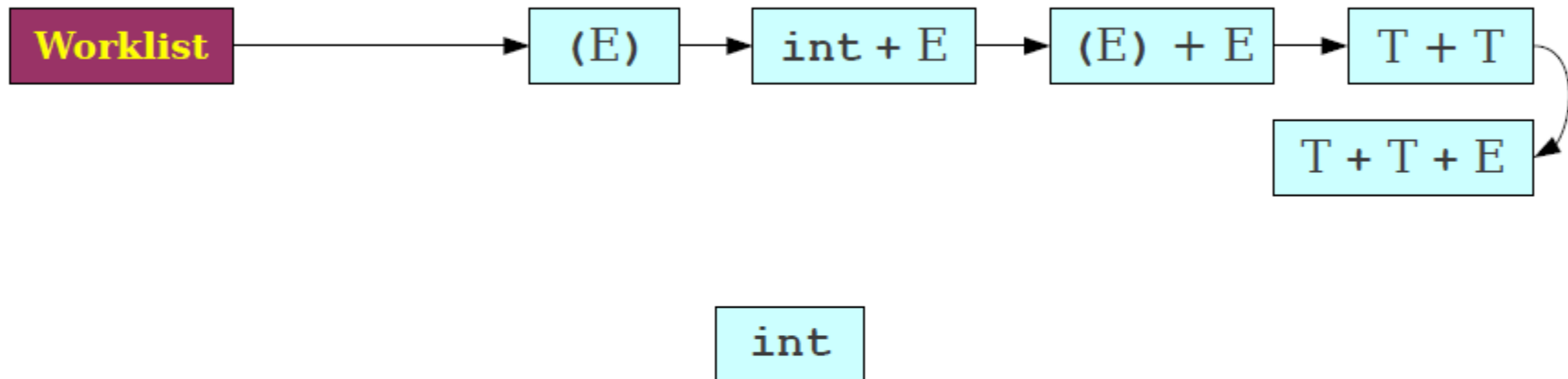
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

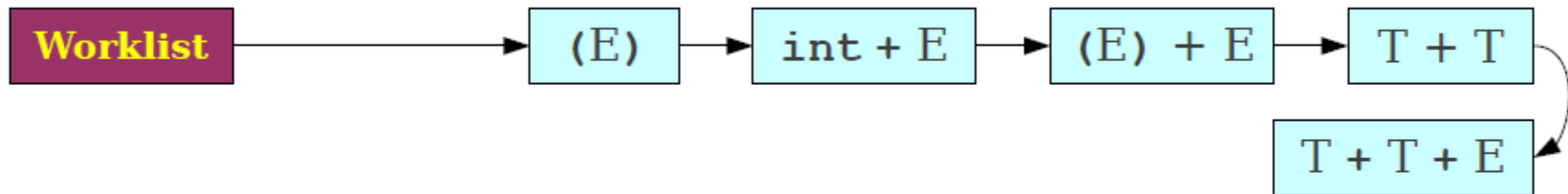
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

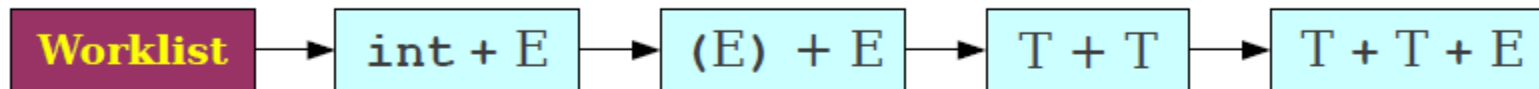
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

Phân tích theo kiểu BFS

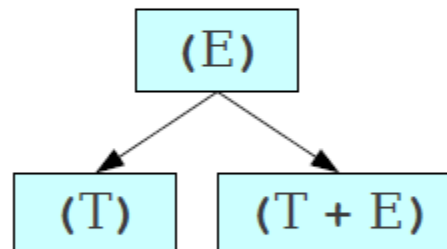
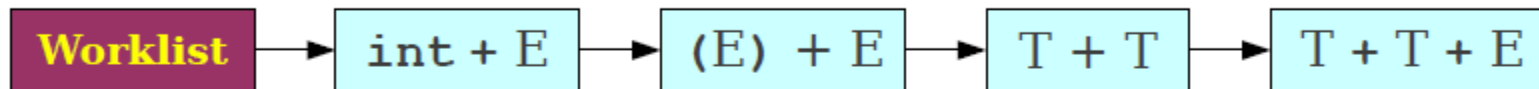


(E)

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

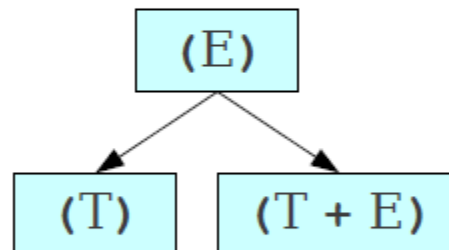
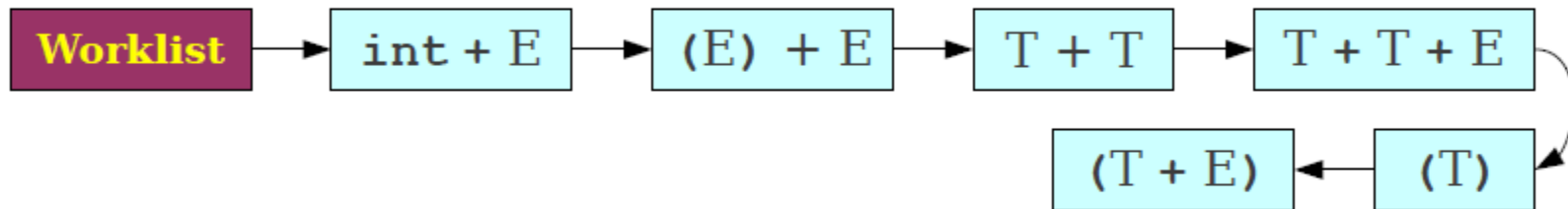
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

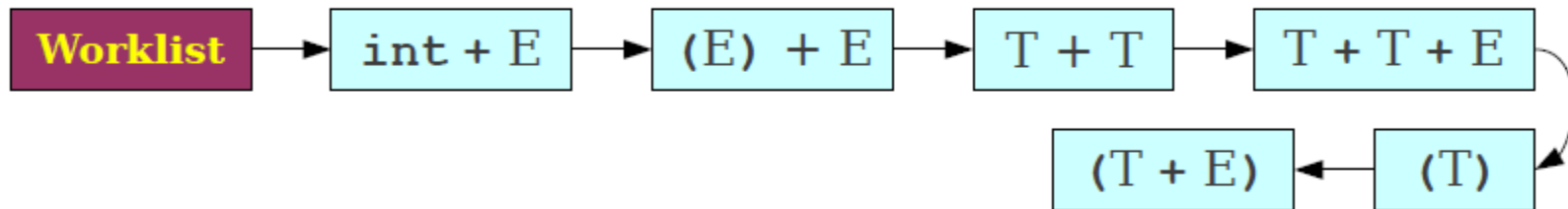
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

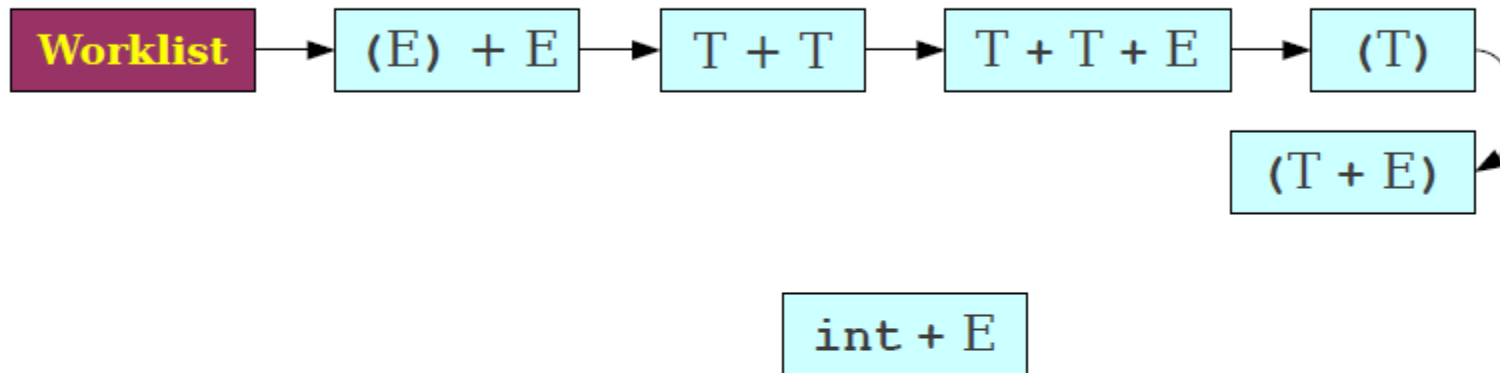
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

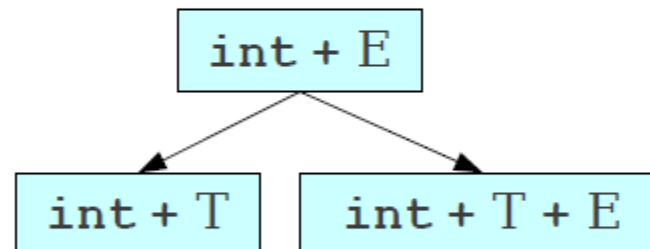
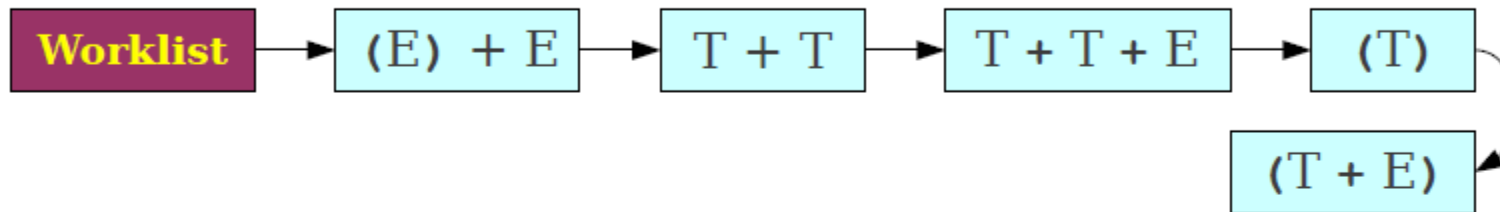
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow int$
 $T \rightarrow (E)$

$int + int$

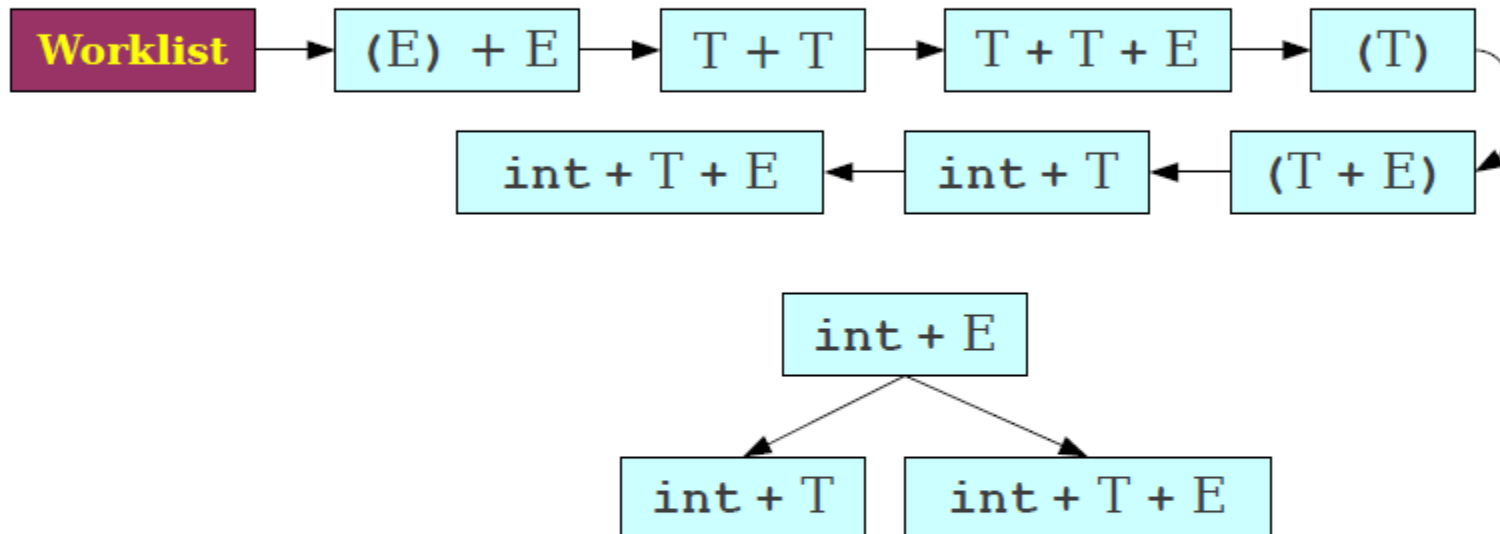
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

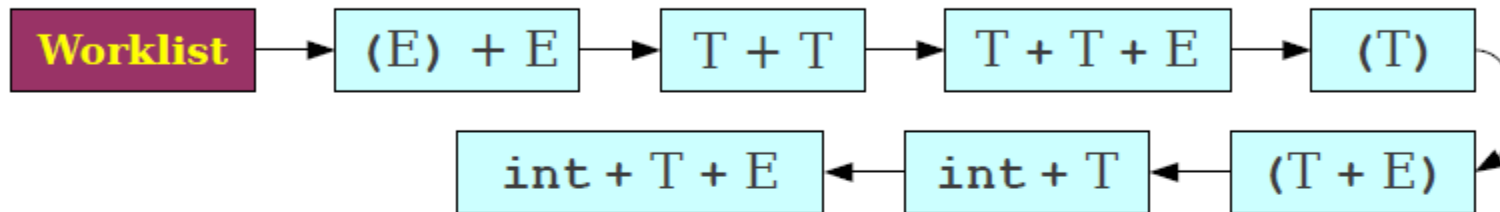
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

$\text{int} + \text{int}$

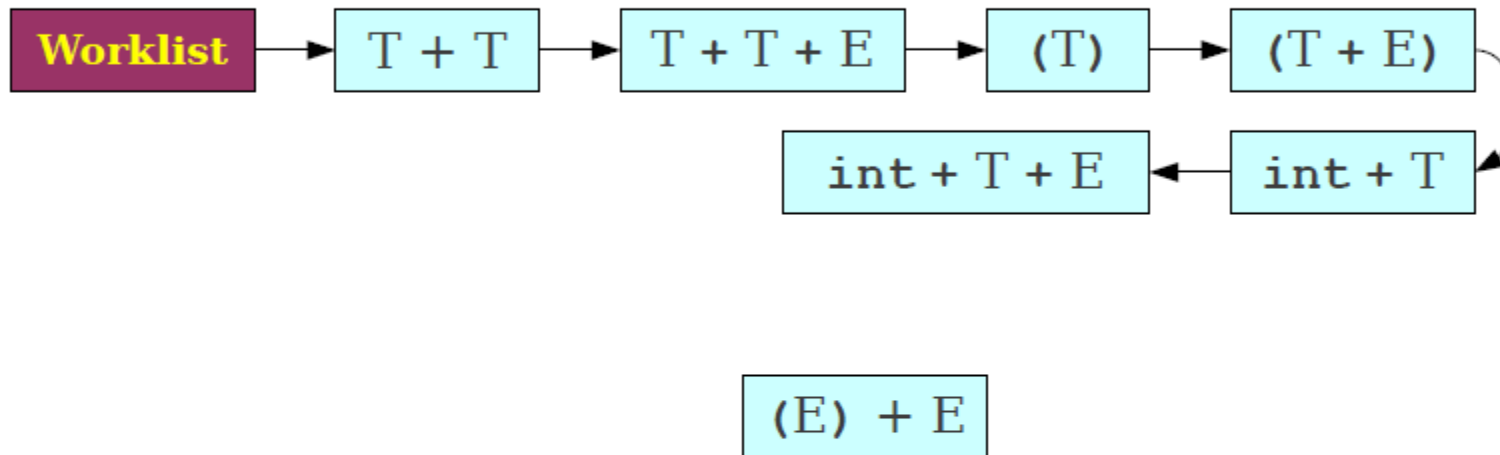
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

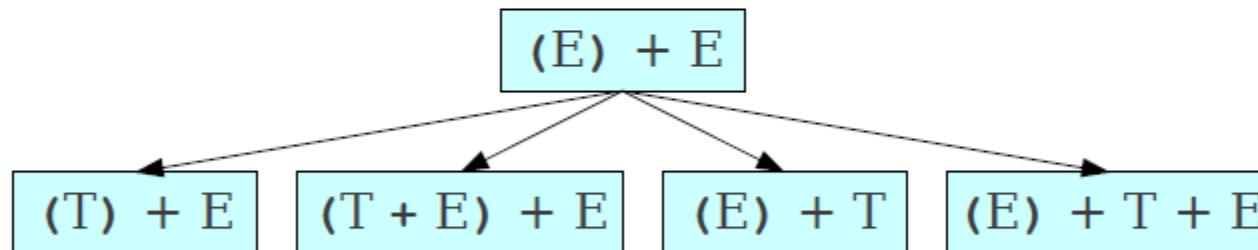
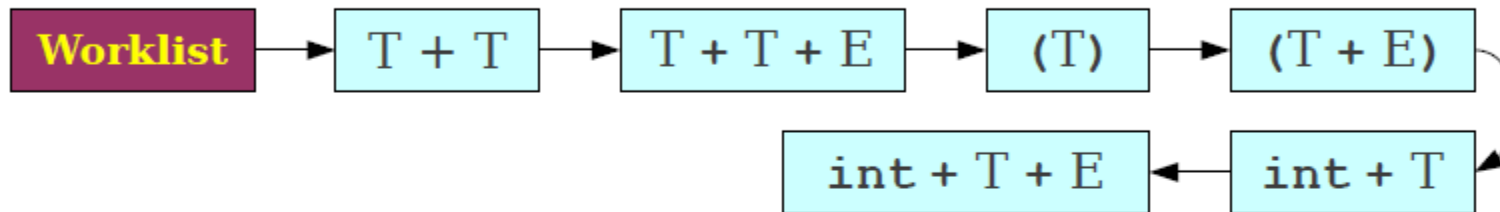
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

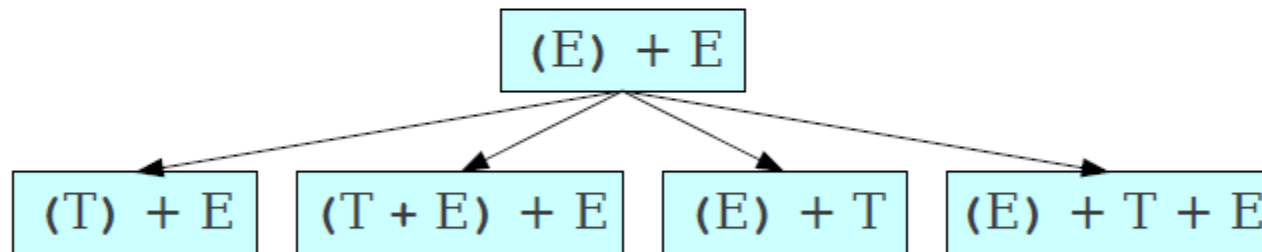
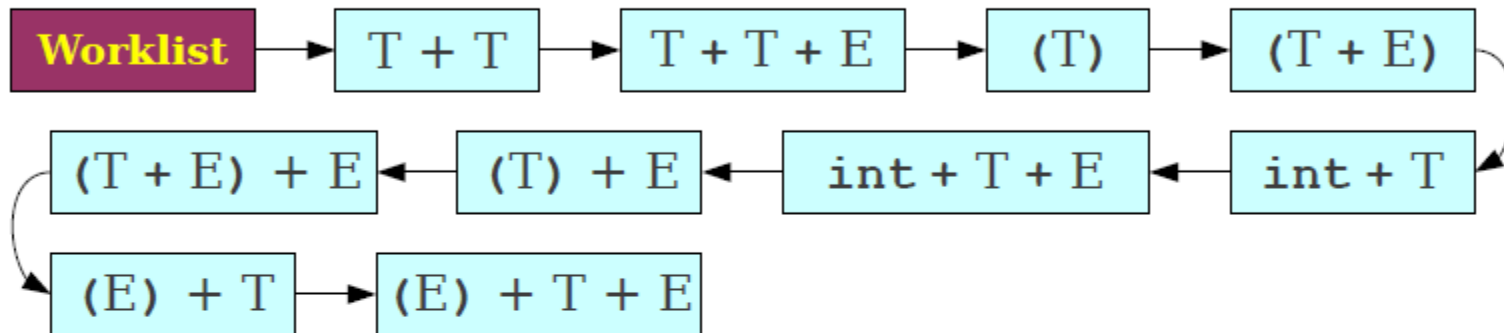
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

$\text{int} + \text{int}$

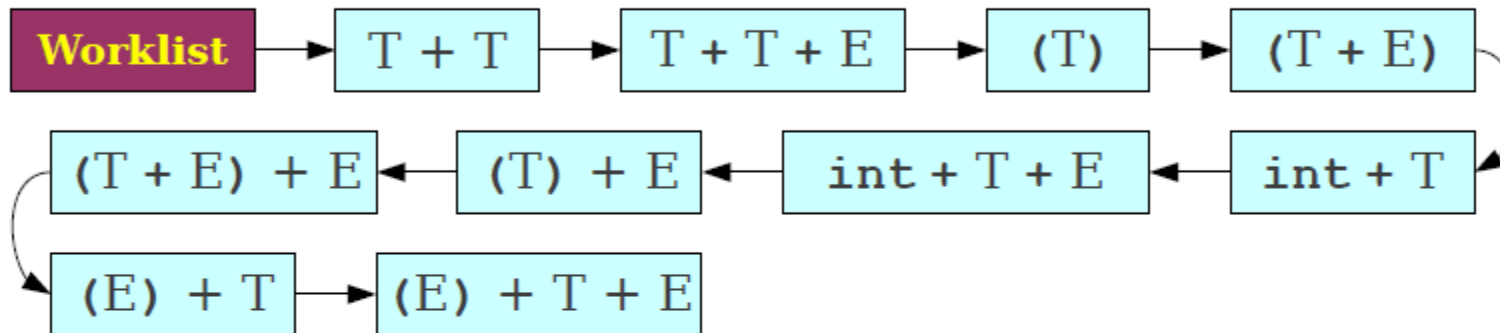
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

$\text{int} + \text{int}$

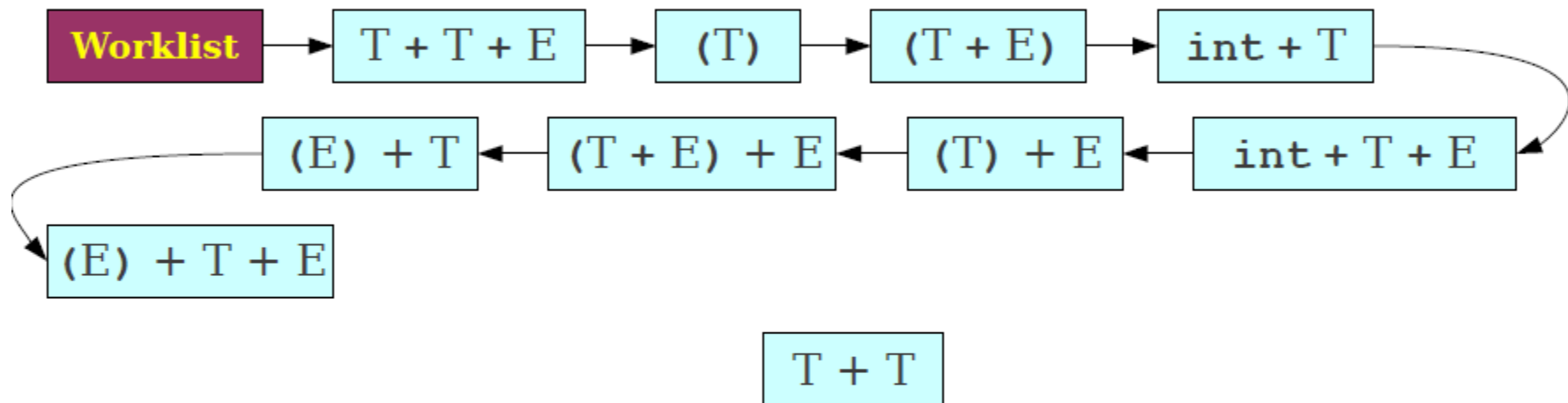
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

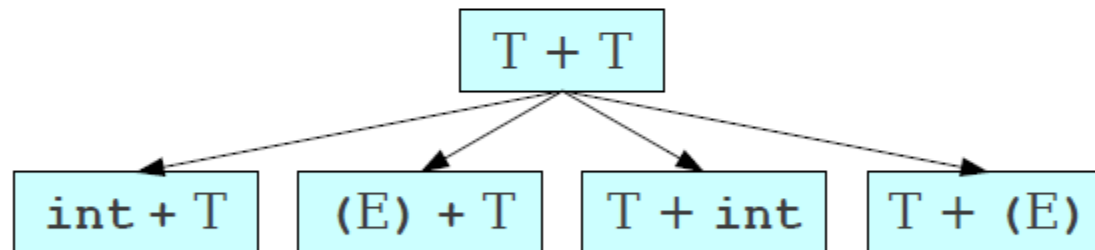
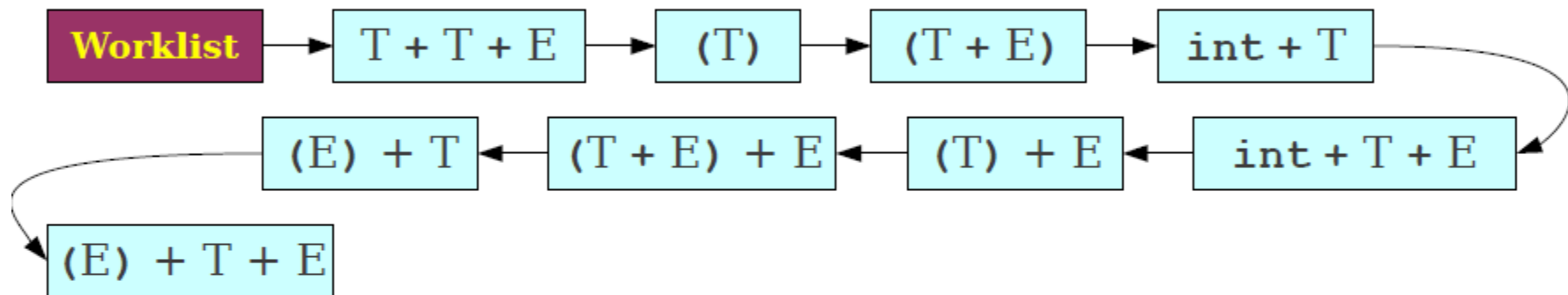
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

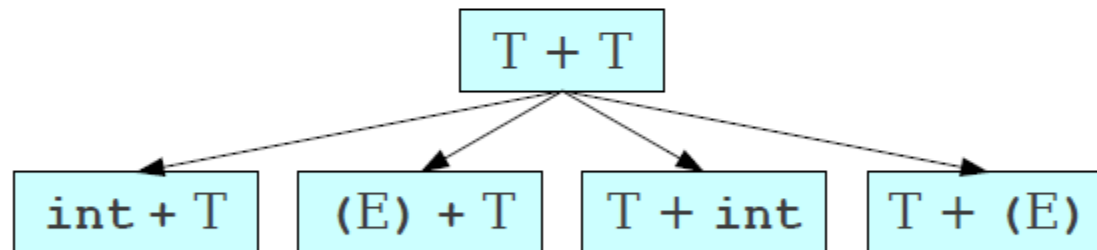
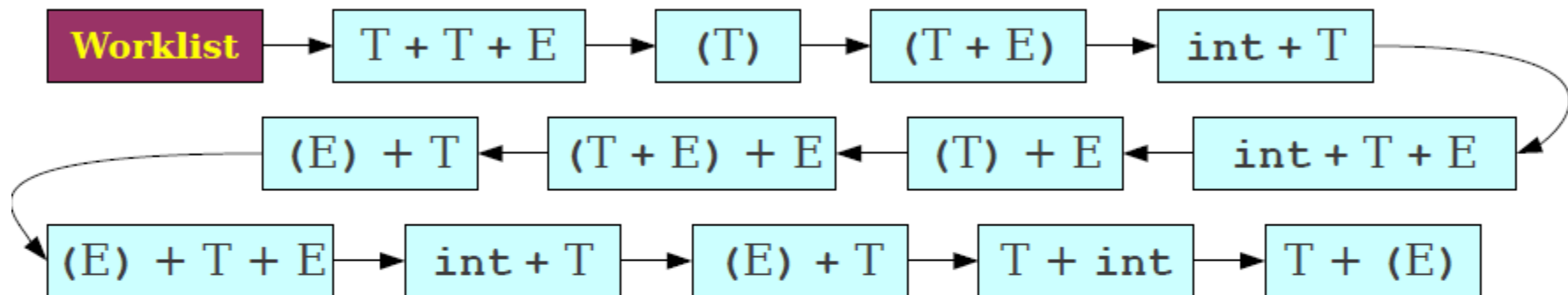
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

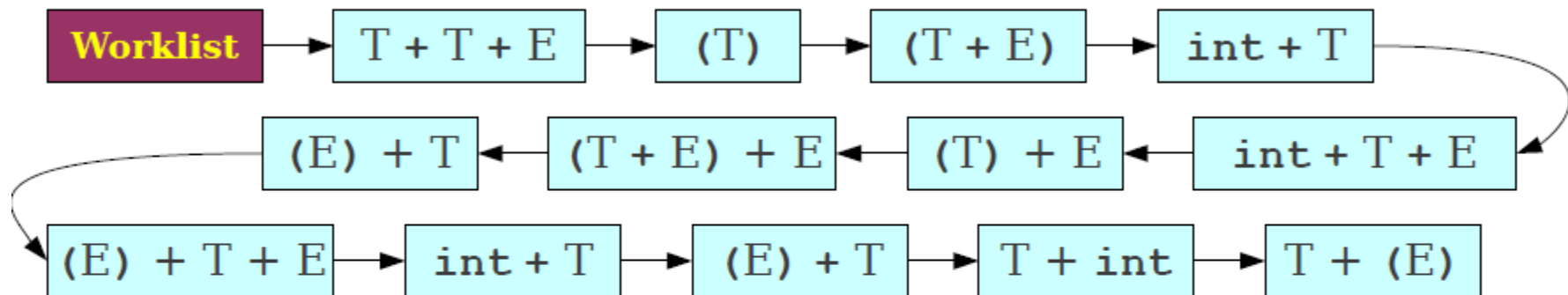
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

$\text{int} + \text{int}$

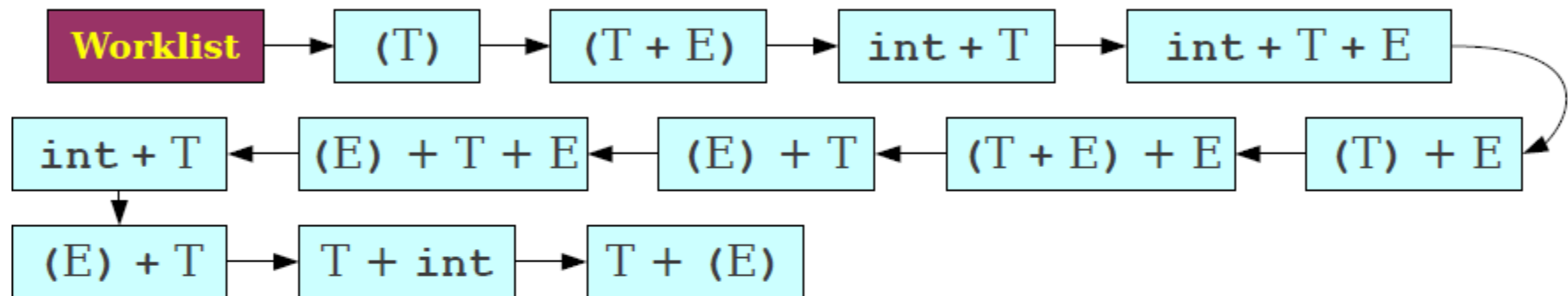
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

Phân tích theo kiểu BFS

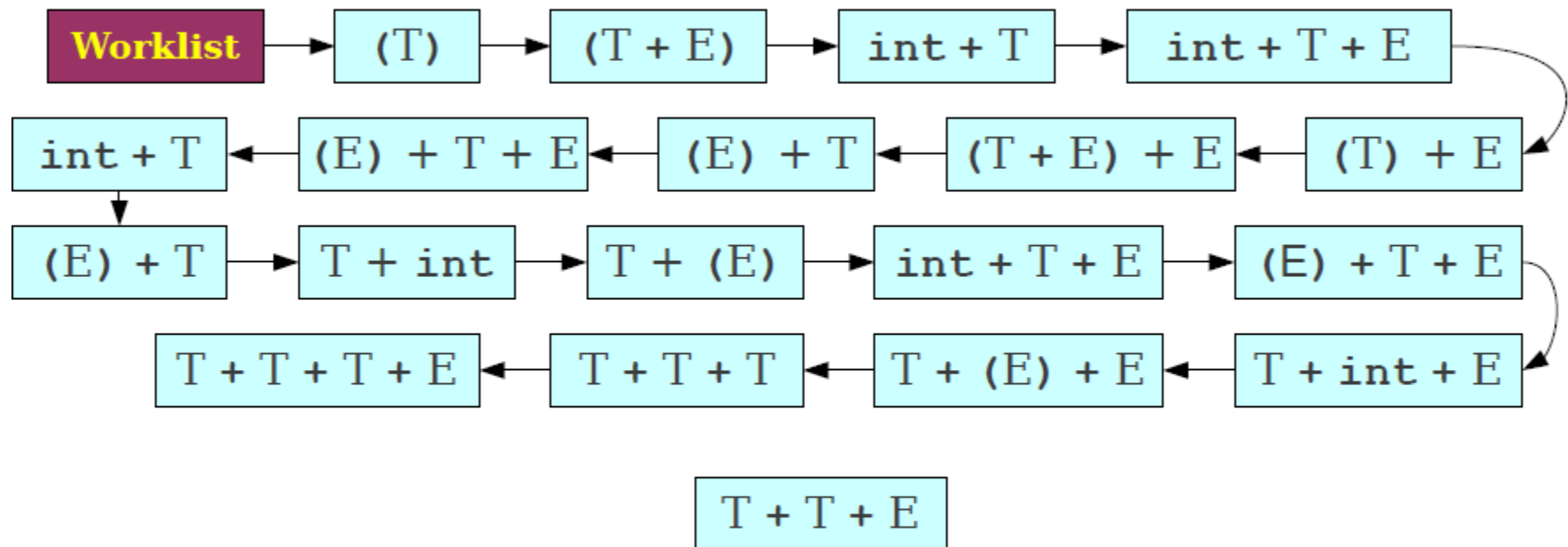


$T + T + E$

$\text{int} + \text{int}$

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

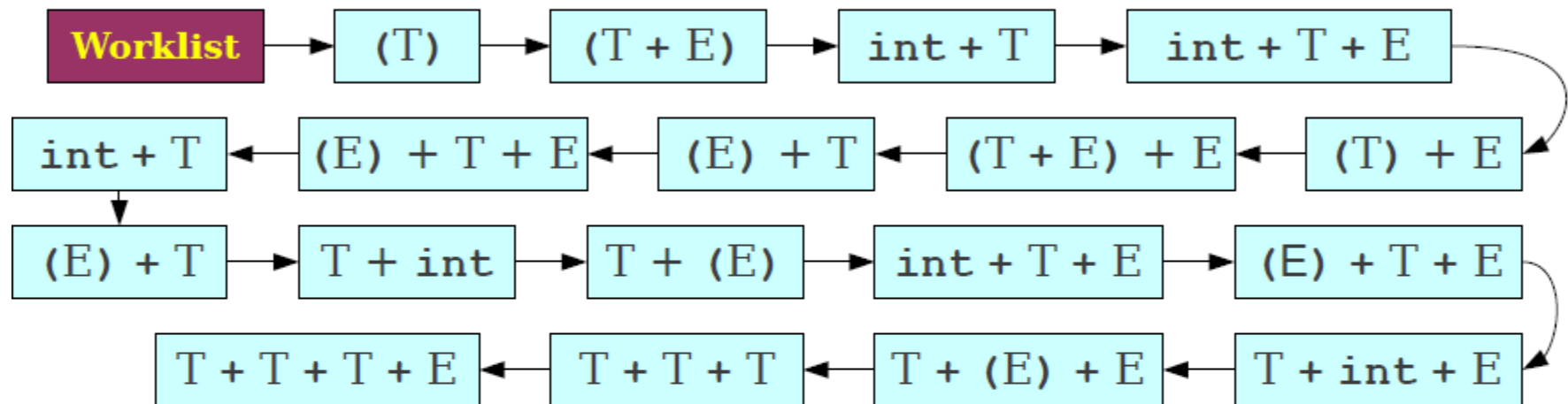
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

$\text{int} + \text{int}$

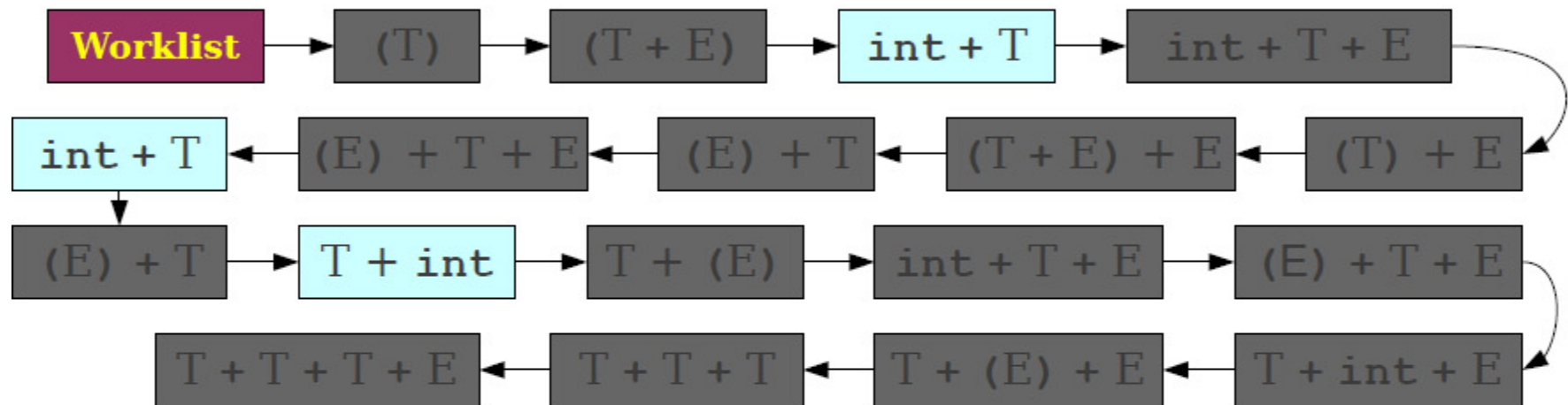
Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

$\text{int} + \text{int}$

Phân tích theo kiểu BFS



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

Phân tích theo kiểu BFS

- Tốn quá nhiều thời gian và bộ nhớ:
 - **Quá nhiều lãng phí:**
 - Sinh ra nhiều câu không có khả năng khớp với đầu vào.
 - Một cách tổng quát, cực kỳ khó để xác định một câu có khả năng khớp với đầu vào hay không – Đây chính là công việc của quá trình phân tích cú pháp !
 - **Hệ số phân nhánh cao (high branching factor)**
 - Mỗi câu có quá nhiều câu con

Giảm thiểu lãng phí

- Giả sử ta đang tìm cách so khớp chuỗi γ .
- Giả sử ta có câu $T = a\omega$, trong đó a là chuỗi các ký tự kết thúc và ω là chuỗi các ký tự kết thúc và chưa kết thúc.
- Nếu ***a không là tiền tố của γ , thì T không thể nào dẫn xuất ra γ .***
- Nếu ta có cách nào đó biết được tiền tố của các ký hiệu kết thúc, ta có thể loại bỏ bớt những câu không có khả năng.

Giảm hệ số phân nhánh

- Nếu một chuỗi có nhiều ký hiệu chưa kết thúc, hệ số phân nhánh sẽ cao (nó có nhiều chuỗi con)
 - Tổng số luật sinh cho mỗi ký hiệu chưa kết thúc nhiều
- Nếu ta có thể giảm thiểu số lượng luật sinh có thể áp dụng, ta có thể giữ được hệ số phân nhánh thấp

Dẫn xuất trái nhất

- Nhắc lại: Dẫn xuất trái nhất là dẫn xuất mà trong đó ta luôn triển khai ký hiệu chưa kết thúc trái nhất.
- Cải tiến giải thuật Phân tích theo kiểu BFS
 - Thực hiện BFS, nhưng chỉ quan tâm đến các **dẫn xuất trái nhất**.
 - Giảm đáng kể hệ số phân nhánh đáng.
 - Tăng khả năng biết được tiền tố của các ký tự chưa kết thúc.
- ● Loại bỏ các câu không có khả năng khớp được với chuỗi đầu vào.
 - Tránh được lãng phí

BFS trái nhất



$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow \text{int}$

$T \rightarrow (E)$

`int + int`

BFS trái nhất

Worklist

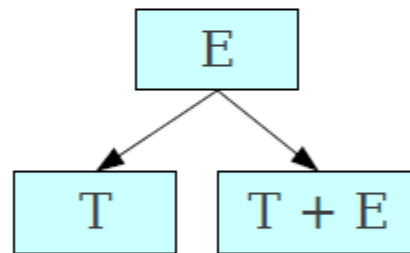
E

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

BFS trái nhất

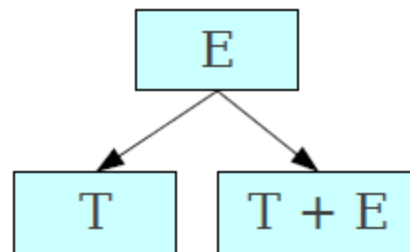
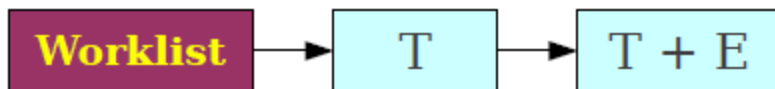
Worklist



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

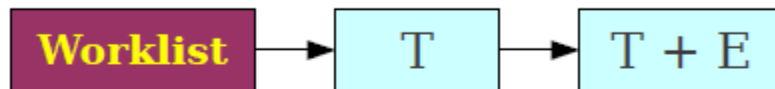
BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

BFS trái nhất



E → **T**

E → **T** + **E**

T → **int**

T → **(E)**

`int + int`

BFS trái nhất

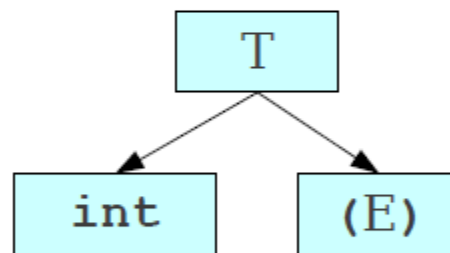


T

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

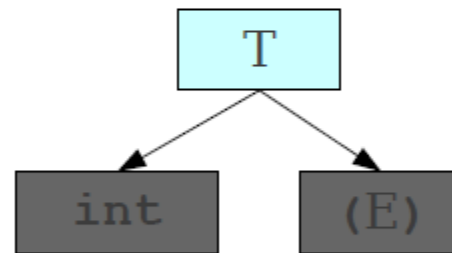
BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

BFS trái nhất



E → **T**

E → **T** + **E**

T → **int**

T → **(E)**

`int + int`

BFS trái nhất

Worklist

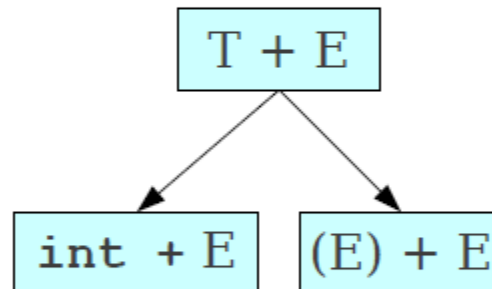
T + E

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

BFS trái nhất

Worklist

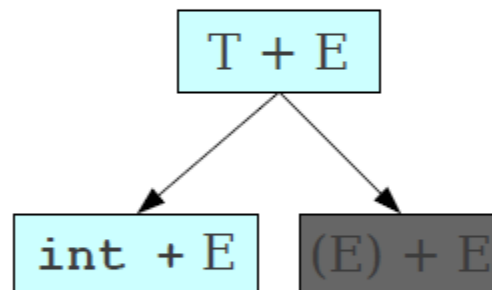


$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

BFS trái nhất

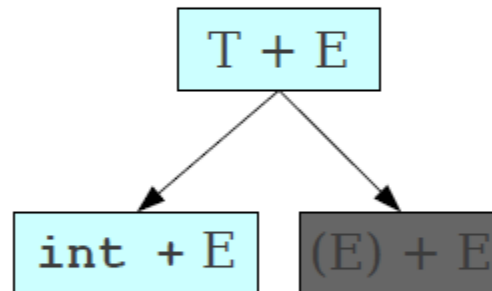
Worklist



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

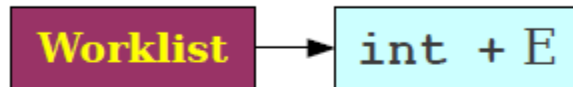
BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

BFS trái nhất



$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow \text{int}$

$T \rightarrow (E)$

int + int

BFS trái nhất

Worklist

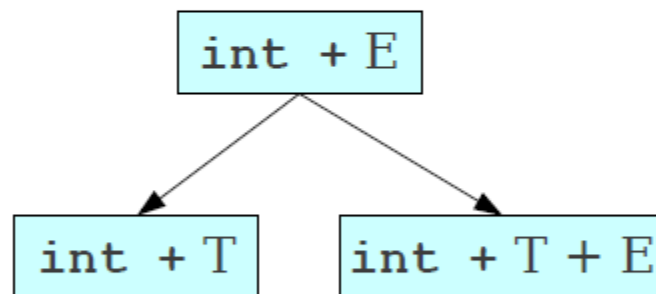
int + E

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

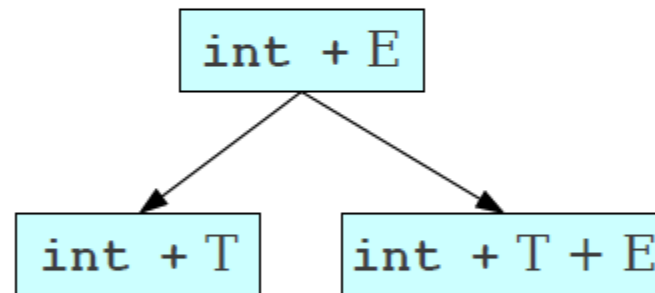
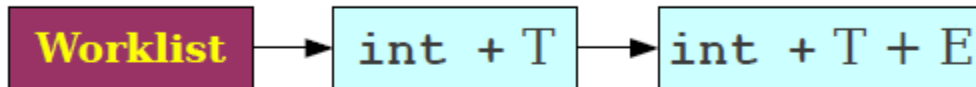
BFS trái nhất

Worklist



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

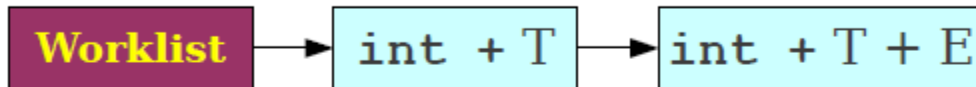
BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

$\text{int} + \text{int}$

BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

BFS trái nhất

Worklist → `int + T + E`

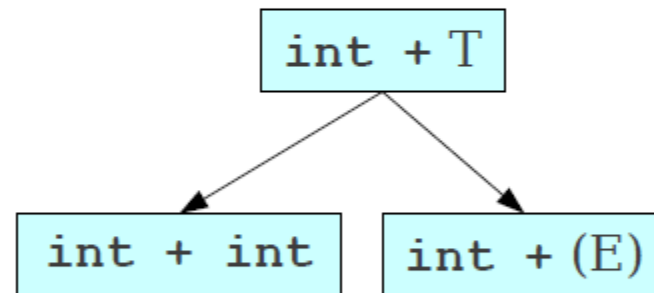
`int + T`

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

BFS trái nhất

Worklist → `int + T + E`

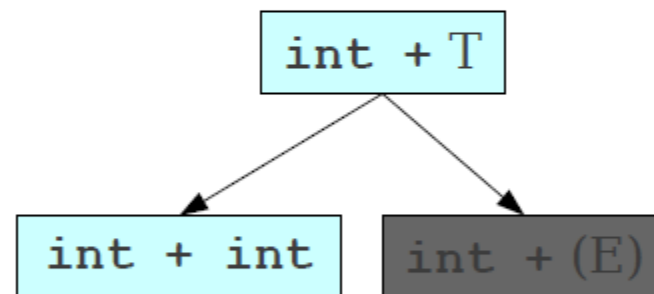


$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

BFS trái nhất

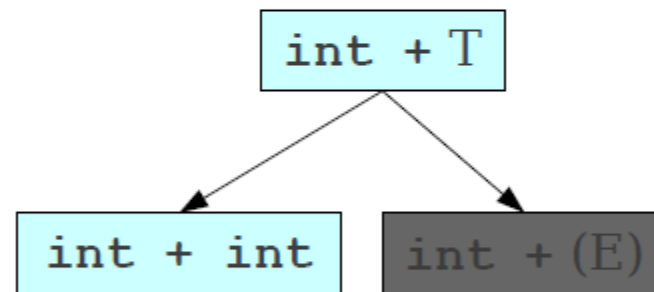
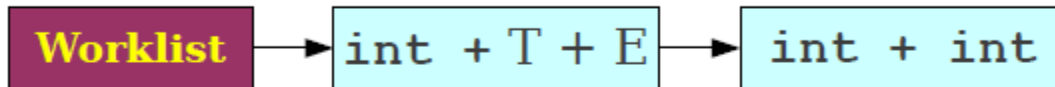
Worklist → `int + T + E`



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

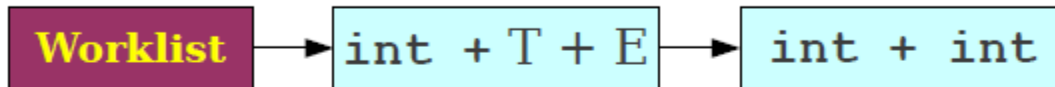
BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

BFS trái nhất



$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow \text{int}$

$T \rightarrow (E)$

`int + int`

BFS trái nhất

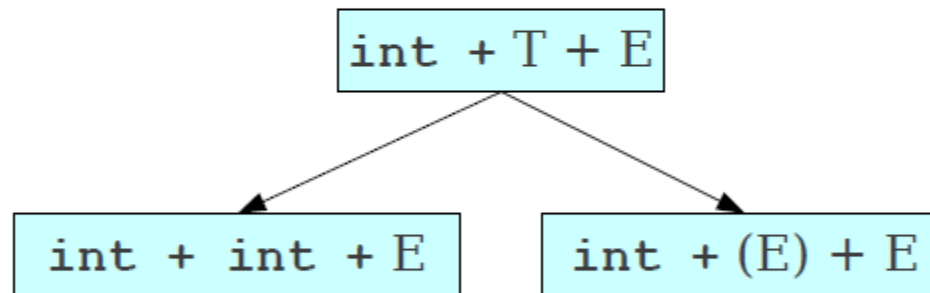
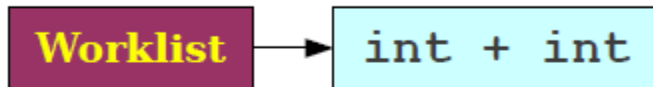
Worklist → int + int

int + T + E

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

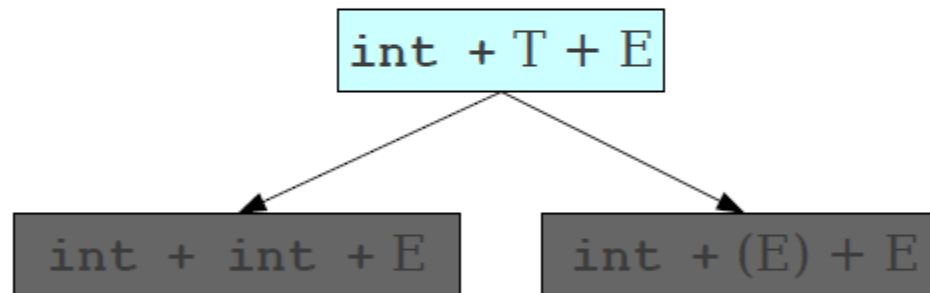
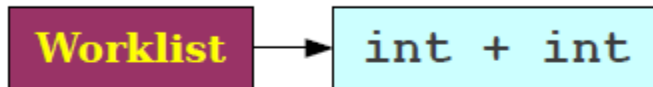
BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

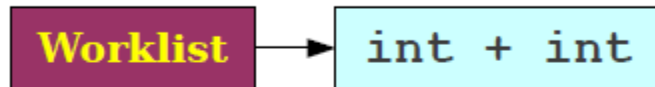
BFS trái nhất



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

BFS trái nhất



$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow \text{int}$

$T \rightarrow (E)$

`int + int`

BFS trái nhất

Worklist

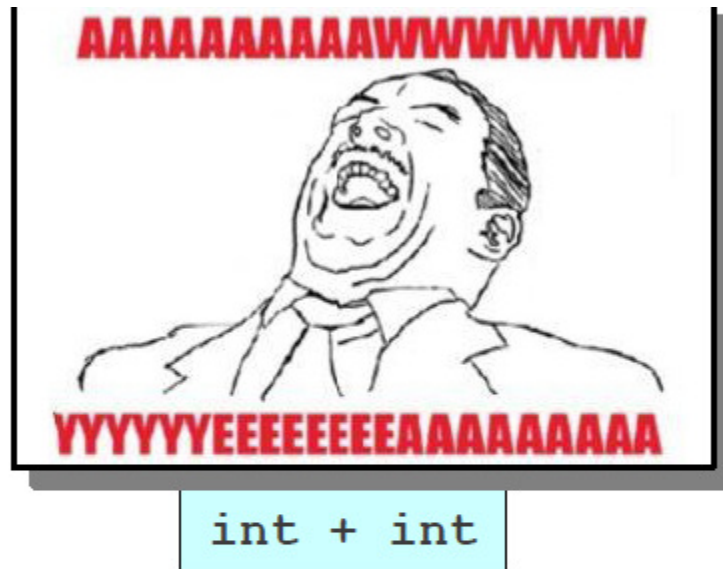
int + int

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

BFS trái nhất

Worklist



$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int + int

BFS trái nhất

- Cải tiến thực sự so với giải thuật BFS gốc
- Sẽ tìm được cách phân tích hợp lệ một chương trình nếu tồn tại.
- Có thể dễ dàng mở rộng để kiểm tra trường hợp chương trình có lỗi cú pháp
- Tuy nhiên, vẫn còn nhiều vấn đề ...

BFS trái nhất

Worklist

$A \rightarrow Aa \mid Ab \mid c$

BFS trái nhất

Worklist

$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất

Worklist

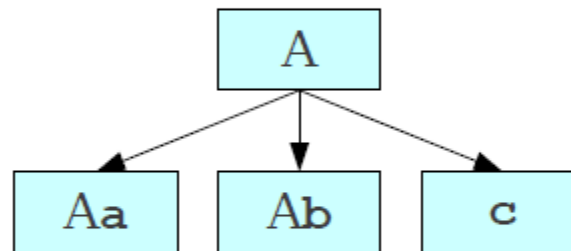
A

$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất

Worklist

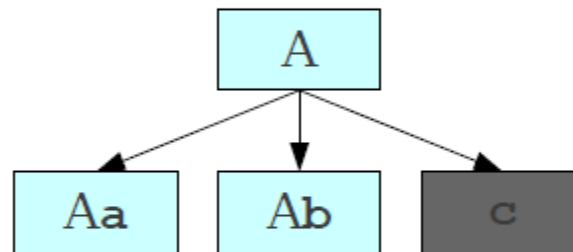


$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất

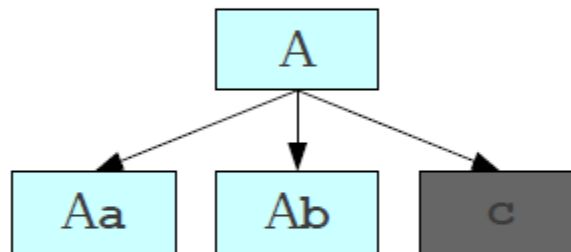
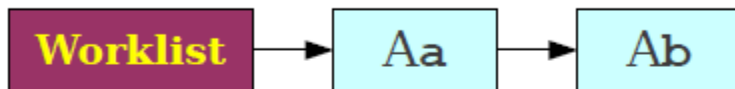
Worklist



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

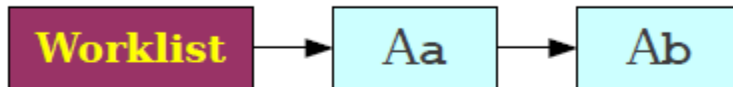
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất



A → **A**a | **A**b | c

caaaaaaaaaa

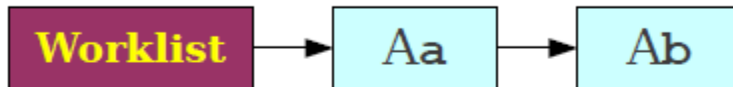
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất

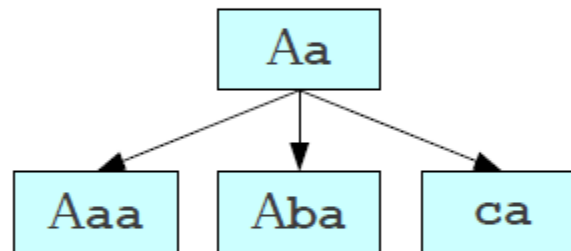


Aa

$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

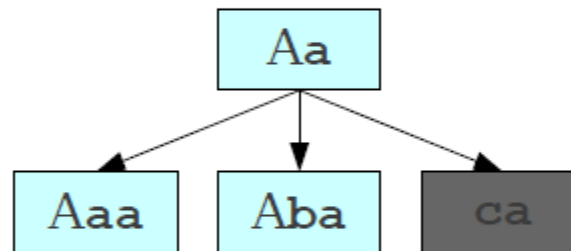
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

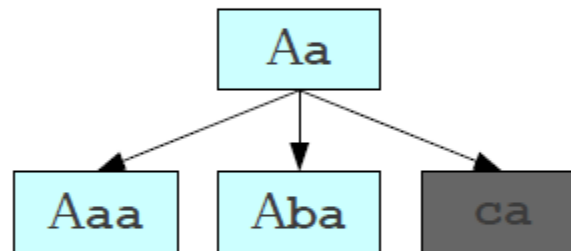
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

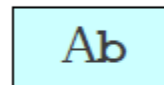
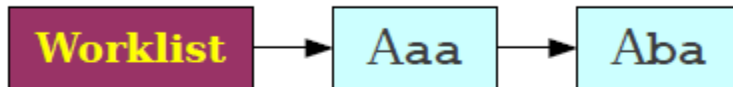
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

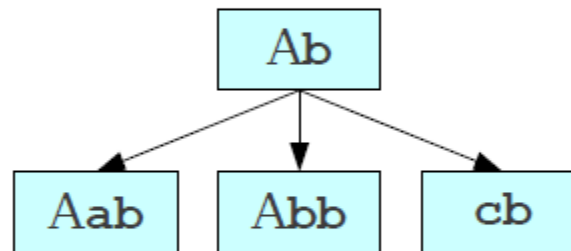
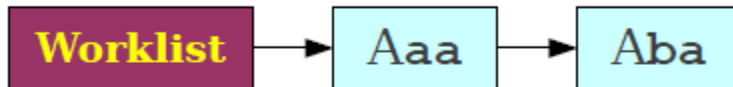
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

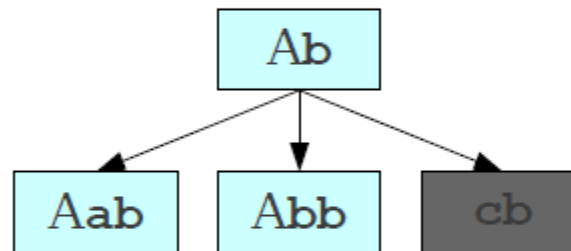
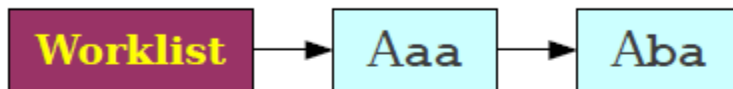
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

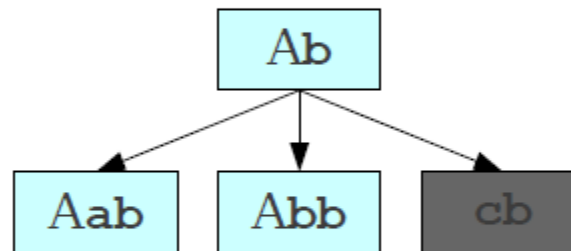
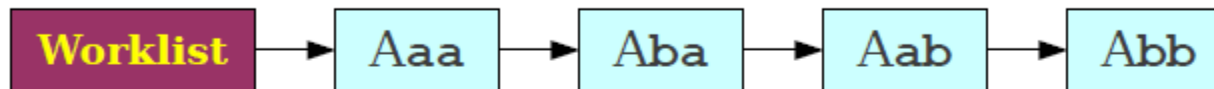
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

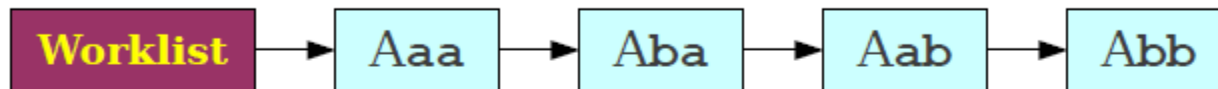
BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất



$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

BFS trái nhất

- Những văn phạm như thế này có thể làm cho độ phức tạp của giải thuật phân tích là lũy thừa.
- Độ phức tạp bộ nhớ cũng lũy thừa.
- Điều gì sẽ xảy ra nếu ta sử dụng một chiến lược tìm kiếm khác?

DFS trái nhất

- Ý tưởng: tìm kiếm theo chiều sâu (DFS).
- Ưu điểm:
 - Ít tốn bộ nhớ: chỉ xem xét mỗi lần 1 nhánh.
 - Hiệu quả cao: chạy rất nhanh trên nhiều văn phạm.
 - Dễ cài đặt: sử dụng đệ quy.

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

DFS trái nhất

E

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T

`int + int`

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T
int

int + int

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T
int

int + int

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T

`int + int`

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T
(E)

`int + int`

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T
(E)

`int + int`

DFS trái nhất

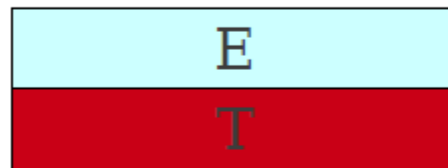
$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T

`int + int`

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$



`int + int`

DFS trái nhất

E

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

`int + int`

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E

`int + int`

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E

int + int

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T

int + int

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + int

int + int

DFS trái nhất

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + int

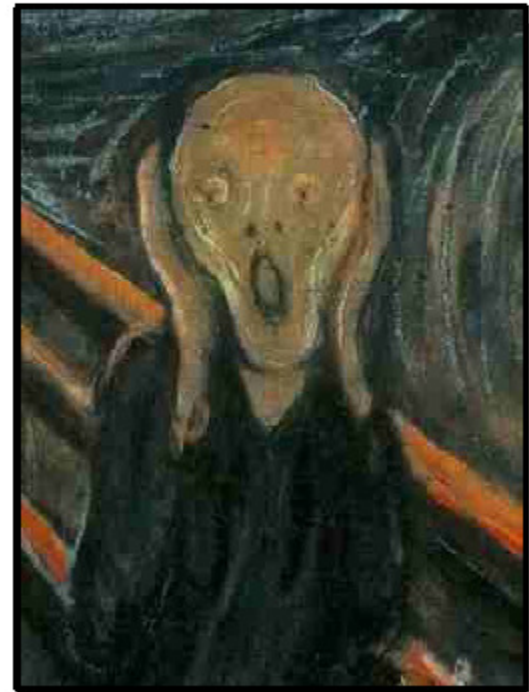
int + int



DFS trái nhất

$A \rightarrow Aa \mid c$

A
Aa
Aaa
Aaaa
Aaaaa



c

Đệ quy trái

- Một ký hiệu chưa kết thúc **A** được gọi là đệ quy trái $\Leftrightarrow$

$$A \Rightarrow^* A\omega$$

với chuỗi ω bất kỳ.

- Giải thuật DFS trái nhất sẽ không chạy được trên văn phạm đệ quy trái.
- May thay, trong nhiều trường hợp ta có thể khử đệ quy trái khá dễ dàng (xem giáo trình).

Tổng kết BFS/DFS trái nhất

- BFS trái nhất làm việc được trên tất cả các loại văn phạm.
- Độ phức tạp thời gian trong trường hợp xấu nhất là lũy thừa.
- Độ phức tạp không gian (bộ nhớ sử dụng) trong trường hợp xấu nhất là lũy thừa.
- Ít sử dụng trong thực tế.
- DFS trái nhất làm việc được trên văn phạm khổng có đệ quy trái.
- Độ phức tạp thời gian trong trường hợp xấu nhất là lũy thừa.
- Độ phức tạp không gian trong trường hợp xấu nhất là tuyến tính.
- Thường được sử dụng trong một số cài đặt bằng phương pháp **đệ quy lùi (recursive descent)**.